Lec-5

09/06/26 10:13 AM

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

K <n

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

110

[1 2 3 4 5 6 7] K=3

5 6 7 1 2 3 4

new-Array

5|6|7|1|2|3|4

[1, 2, 3, 4, 5, 6, 7] → n

K=1 7 1 2 3 4 5 6

K=2 6 7 1 2 3 4 5

K=3 5 6 7 1 2 3 4

K=4 4 5 6 7 1 2 3

K=5 3 4 5 6 7 1 2

K=6 2 3 4 5 6 7 1

K=7 1 2 3 4 5 6 7

K=8 7 1 2 3 4 5 6

K=9 6 7 1 2 3 4 5

K=10 5 6 7 1 2 3 4

K=11 4 5 6 7 1 2 3

K=12

K=N

no K=110

15×7=105

110%7=5

K=K%N

7|1|2|3|4|5|6

0 1 2 3 4 5 6

7|1|2|3|4|5|6

0 1 2 3 4 5 6

for(i=n-2 i>=0 i--){
arr[i+1]=arr[i];
}

(i) item=arr[6]→7

i=5 arr[6]=arr[5]

i=4 arr[5]=arr[4]

i=3 arr[4]=arr[3]

i=2 arr[3]=arr[2]

i=1 arr[2]=arr[1]

i=0 arr[1]=arr[0]

(ii) arr[0]=item

(ii) (i)

4 3 2 1 | 7 6 5

1|2|3|4|5|6|7

K=3

5 6 7 1 2 3 4

(i) N-K | last K (ii)

1|2|3|4|5|6|7

0 1 2 3 4 5 6

i j n-1

K=3

Starting N-K

(i) [i=0 j=N-K-1]

(ii) [i=n-k j=n-1]

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is `0` because there are no elements to the left. This also applies to the right edge of the array.

Return *the **leftmost pivot index***. If no such index exists, return `-1`.

Example 1:

```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

Example 2:

[1 7 3 6 5 6]

0 1 2 3 4 5

Pi=0 L=0 R=27 X

Pi=1 L=1 R=20 X

Pi=2 L=8 R=17

Pi=3 L=11 R=11

[1 7 3 6 5 6]

0 1 2 3 4 5

```
public static int leftSum(int[] arr, int si,int ei) {
    int sum=0;
    for (int i = si; i <ei; i++) {
        sum+=arr[i];
    }
    return sum;
}
```

i=0 → [0,0], [1,5]
Left R

i=1 [0,1] [2,5]

i=2 [0,2] [3,5]

i=3 [0,3] [4,5]

i=4 [0,4] [5,5]

[1 7 3 6 5 6]

0 1 2 3 4 5

Left 0|1|8|11|17|22

0 1 2 3 4 5

Right 17|14|11|6|0

0 1 2 3 4 5

L[i]=L[i-1]+arr[i-1]

R[i]=R[i+1]+arr[i+1]

Left[0]=0

i=1 Left[1]= Left[0]+arr[0]

i=2 Left[2]= Left[1]+arr[1]

i=3 Left[3]= Left[2]+arr[2]

i=4 Left[4]= Left[3]+arr[3]

i=5 Left[5] = Left[4]+arr[4]

n-1 R[5] → R[N-1]=0

i=4 R[4]= R[5]+arr[5]

i=3 R[3] = R[4]+arr[4]

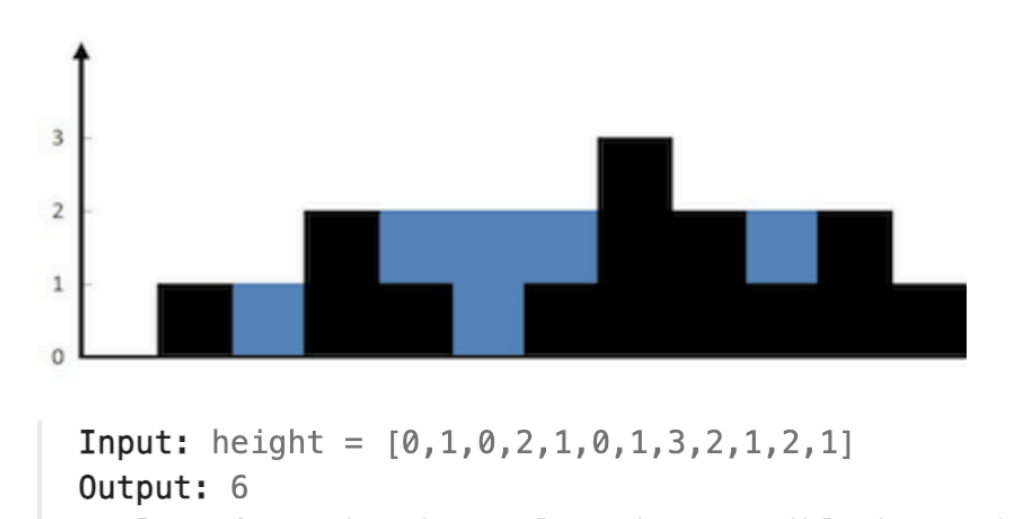
i=2 R[2]= R[3]+arr[3]

i= R[1]= R[2]+arr[2]

i=

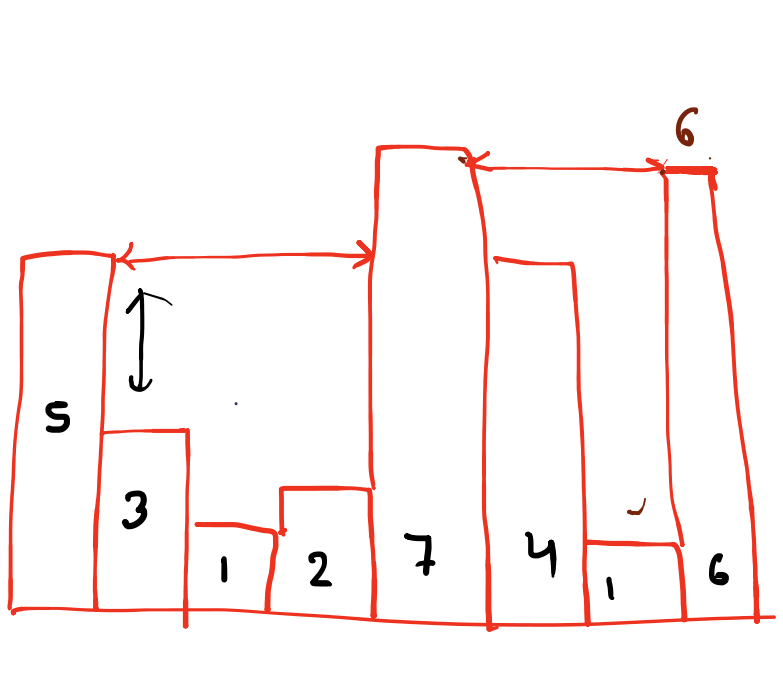
Given `n` non-negative integers representing an elevation map where the width of each bar is `1`, compute how much water it can trap after raining.

Example 1:



```
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.
```

2+4+3+2+5=16 ✓



0 1 2 3 4 5 6 7

[5, 3, 1, 2, 7, 4, 1, 6]

5|5|5|5|7|7|7|7

0 1 2 3 4 5 6 7

7|7|7|7|7|6|6|6

0 1 2 3 4 5 6 7

i=0 → min[L[0],R[0]]-arr[0] = (5,7)-5=0

i=1 → min[L[1],R[1]]-arr[1]= (5,7)-3 = 2

i=2 → min[L[2],R[2]]-arr[2]= (5,7)-1 = 4

i=3 → min[L[3],R[3]]-arr[3] = (5,7)-2 = 3

i=4 → min[L[4],R[4]]-arr[4] = (7,7)-7 = 0

i=5 → min(L[5],R[5])-arr[5]= (7,6)-4 = 2

i=6 → min[L[6],R[6]]-arr[6] = (7,6)-1 = 5

i=7 → min[L[7],R[7]]-arr[7] = (7,6)-6 = 0

for(i=0 i<n i++){
sum = sum+{min(L[i], R[i]) - arr[i]}
}

0 1 2 3 4 5 6 7

[5, 3, 1, 2, 7, 4, 1, 6]

7|7|7|7|7|6|6|6

0 1 2 3 4 5 6 7

5|5|5|5|7|7|7|7

0 1 2 3 4 5 6 7

Left[0]=arr[0]

Left[i]= max(Left[i-1],arr[i])

Left[1]= max(Left[0],arr[1])

Left[2]= max(Left[1],arr[2])

Left[3]= max(Left[2],arr[3])

Left[4]= max Left[3], arr[4])

Left[5]= max Left[4], arr[5]

Left[6] = max Left[5], arr[6]

Left[7]= max Left[6], arr[7])

R[7]=arr[7]

R[i]=R[i+1],arr[i]
max

R[6]= max(R[7], arr[6])

R[5] = max(R[6], arr[5])

R[4]= max(R[5], arr[4])

R[3] = max(R[4], arr[3])

R[2] = max(R[3], arr[2])

R[1] = max(R[2], arr[1])

R[0] = max R[1], arr[0])

Given an integer array `nums`, return *an array* `answer` *such that* `answer[i]` *is equal to the product of all the elements of* `nums` *except* `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in `O(n)` time and without using the division operation.

2|3|6|4

2|3|6|4

72|48|24|36

2|3|6|4

0 1 2 3

1|2|6|36 Left

0 1 2 3

72|24|4|1

72|48|24|36 =

i=

2|3|6|4

1|24|4|1

0 1 2 3

1|2|6|36

Left[0]=1

Left[1]= Left[0]×arr[0]

Left[2]= Left[1]×arr[1]

Left[3]= Left[2] ×arr[2]

R[2]= R[3]×arr[3]

R[1]= R[2]×arr[2]

ans=-∞ 2 5 8 11

0 1 2 3 4 5

[2, 3, -7, 4, -1, 8]

[0-0] 2 2

[0-1] 2 3 5

0-2 2 3 -7 -2

0-3 2 3 -7 4 = 2

0-4 2 3 -7 4 -1 = 1

0-5 2 3 -7 4 -1 8 9

1-1 3 3

1-2 3 -7 -4

1-3 3 -7 4 0

1-4 3 -7 4 -1 -1

1-5 3 -7 4 -1 8 7

2-2 -7 -7

2-3 -7 4 -3

2-4 -7 4 -1 -4

2-5 -7 4 -1 8 4

3-3 4 4

3-4 4 -1 3

3-5 4 -1 8 11

4-4 -1 -1

4-5 -1 8 7

5-5 8 8

for(i=0 i<n i++){
int sum=0
for(j=i j<n j++){
sum += arr[j]
ans = max(ans, sum)
}
}

0 1 2 3 4 5

[2, 3, -7, 4, -1, 8]

```
public static int SubarraySum(int[] arr) {
    int ans = Integer.MIN_VALUE;
    int sum = 0;
    for (int i = 0; i < arr.length; i++) {
        sum += arr[i];
        ans = Math.max(ans, sum);
    }
    return ans;
}
```

ans=2 5

0-0 2

0-1 2 3

0-2 2 3 -7

0-3 2 3 -7 4

0-4 2 3 -7 4 -1

0-5 2 3 -7 4 -1 8

sum=2+3
=5-7=-2
=-2+4=2

ans=2 5

```
public static void main(String[] args) {
    // TODO Auto-generated method stub
    int[] arr = { -2, 1, 3, 4, -1, 2, 1, -5, 4 };
    System.out.println(SubarraySum(arr));
}

public static int SubarraySum(int[] arr) {
    int ans = Integer.MIN_VALUE;
    int sum = 0;
    for (int i = 0; i < arr.length; i++) {
        sum += arr[i];
        ans = Math.max(ans, sum);
    }
    return ans;
}
```

if(sum<0){
sum=0
}

-2

0 1 2 3 4 5

[2, 3, -7, 4, -1, 8, -13, 2, -1, 5]

ans=-∞ 2 8 11

sum= sum+arr[0]
= 2+arr[1]
=2+3
=5

sum= 5+arr[2]
= 5-7=-2

sum=4
sum=3+8
= 11-13
=-2

sum=2
=2+1=1 15
=6